VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Machine learning

Final Report

Comparative Study of Machine Learning and Deep Learning for Hate Speech Classification

---

Advisor(s): Trương Vĩnh Lân

Student(s): Trần Quốc Bảo Long ID 2252453
Nguyễn Bình Nguyên ID 2252545
Đặng Ngọc Phú ID 2252617
Đặng Duy Nguyên ID 2352821
Nguyễn Văn Hoàng Phát ID 2352896

HO CHI MINH CITY, MAY 2026

Contents

1 Introduction

1.1 Motivation

The rapid expansion of social media platforms has transformed how people communicate and share information online. While these platforms enable open discussion and global connectivity, they have also become channels for the spread of harmful content such as hate speech and offensive language. Detecting such content manually is increasingly difficult due to the massive volume of user-generated text. In recent years, techniques from machine learning and deep learning have shown strong potential for automatically identifying toxic language in online communication.

1.2 Project Objective

This project develops and evaluates models for detecting hate speech in short social-media texts. Specifically, it conducts a **comparative study** between traditional machine learning approaches and modern deep learning methods in order to assess their effectiveness and trade-offs on a binary toxicity-classification task.

The study is built around five distinct feature representations, fed into seven traditional classifiers (with hyperparameter tuning) and two deep-learning architectures (a fine-tuned DistilBERT and a BiLSTM with GloVe embeddings). The total experimental matrix produces 27 baseline results, three tuned results, and two deep-learning results, all evaluated under a unified protocol.

1.3 Report Roadmap

The rest of this report is structured as follows:

- Section 2 introduces the dataset and explains why it is well-suited to a comparative study of this kind.
- Section 3 reports the exploratory data analysis (EDA), with particular emphasis on the severe class imbalance.
- Section 4 describes the text preprocessing pipeline.
- Section 5 details the five feature extraction methods.
- Section 6 documents the train/test split and saved artefacts.

- Section 7 introduces the seven classifiers and the feature–model compatibility constraints.
- Sections 8 to 10 report the baseline grid, confusion-matrix and ROC analysis, and hyperparameter tuning.
- Sections 11 and 12 cover the deep-learning pipeline and the unified ML-vs-DL comparison.
- Sections 13 to 15 provide the extended analyses: error analysis, model interpretability, and a comparison between SMOTE and class-weight balancing.
- Section 16 summarises findings, limitations, and future work, and Section 17 gives the task-allocation table.

2 Dataset Description

2.1 Source and Basic Properties

The dataset is the **Twitter Toxic Tweets** corpus, hosted on Kaggle at `kaggle.com/datasets/um` It contains 31,962 real-world Twitter posts manually labelled for toxic or non-toxic content. Table 2.1 summarises its basic properties.

Table 2.1: Basic dataset statistics

Property	**Value**
Total samples	31,962
Columns	`id`, `tweet`, `label`
Missing values	None
Classes	0 = Non-toxic, 1 = Toxic
Source	Kaggle (Twitter Toxic Tweets, Umitka)

2.2 Why This Dataset Is Suitable

This dataset is appropriate for the project for several reasons. First, it focuses on *short, real-world social-media text*, which is noisy, informal, and highly variable in writing style. Compared with longer and more formal text sources, tweets contain abbreviations, slang,

hashtags, mentions, misspellings, and expressive punctuation. These characteristics make toxic-tweet detection more challenging — and more realistic — for practical machine-learning applications in online content moderation.

Second, the dataset size (31,962 samples) is large enough to yield statistically meaningful comparisons but small enough to remain tractable for both traditional ML and deep-learning models on consumer GPU hardware, which directly supports the comparative goal of this project.

Third, because the labels are binary, the task is more focused than multi-class or multi-label toxicity detection, allowing the group to concentrate on a clean comparison between baseline ML approaches and neural models without label-design ambiguity.

2.3 Expected Challenges

From a machine-learning perspective, three main challenges are anticipated:

1. **Heavy preprocessing required.** Social-media text is unstructured and contains URLs, mentions, hashtags, emojis, and non-standard spellings.

2. **Sparse / context-dependent toxic signals.** Toxic meaning often depends on wording patterns, sarcasm, or identity-related terms, which are difficult for simple bag-of-words features to capture.

3. **Severe class imbalance.** As shown in Section 3, the dataset is heavily skewed toward non-toxic tweets. Raw accuracy is therefore an unreliable metric — F1-score, precision, and recall for the toxic class must be reported instead.

3 Exploratory Data Analysis

3.1 Class Distribution

Table 3.1 shows the label distribution. The dataset exhibits a **severe class imbalance**: 93% of tweets are non-toxic and only 7% are toxic. This imbalance directly motivates the choice of evaluation metrics (F1-macro, F1-toxic) and the imbalance-handling strategies (`class_weight='balanced'`, SMOTE) used later in the project.

Table 3.1: Class distribution

Label	Class	Count	Proportion
0	Non-toxic	29,720	93.0%
1	Toxic	2,242	7.0%
	Total	31,962	100%

3.2 Tweet Length Analysis

Table 3.2 provides descriptive statistics of tweet length measured in characters and words. Tweets are relatively short, consistent with the Twitter character limit of the period. Toxic tweets tend to be slightly longer on average than non-toxic ones, suggesting that hateful content may require more words to express.

Table 3.2: Tweet length statistics (raw text)

Metric	Mean	Std	Min	25%	Median	Max
Character length	84.7	29.5	11	63	88	274
Word count	13.2	5.5	3	9	13	34

3.3 EDA Visualisations

The notebook produces five EDA visualisations, all saved to `reports/figures/`:

- **Class distribution** — bar chart and pie chart highlighting the 93%/7% imbalance.
- **Tweet length distributions** — KDE and box plots of character length and word count, broken down by class.
- **Top-20 most frequent words per class** — horizontal bar charts computed on the cleaned corpus (after preprocessing).
- **Word clouds per class** — visual summary of dominant vocabulary for non-toxic (blue palette) and toxic (red palette) tweets.
- **Feature dimensionality overview** — bar chart comparing the output dimensions of each of the five feature extraction methods.

4 Text Preprocessing Pipeline

4.1 Design Rationale

Two cleaning variants are produced for each tweet, because different downstream models have different input requirements:

- `tweet_clean` — fully cleaned text, used for Bag-of-Words, TF-IDF, Char n-gram TF-IDF, and GloVe.
- `tweet_bert` — lightly cleaned text, used for DistilBERT. Stopword removal and lemmatisation are skipped so that the model sees natural sentence structure, which contextual transformers rely on.

4.2 Full Cleaning Pipeline (`tweet_clean`)

The following steps are applied in order:

1. **Emoji to text** — emojis are converted to their text descriptions using the `emoji` library (e.g. → `angry_face`). This preserves sentiment signal that would otherwise be lost.
2. **URL removal** — HTTP/HTTPS URLs and bare `www.` links are replaced with a space.
3. **Mention removal** — Twitter `@username` tokens are removed (they carry no topical information).
4. **Hashtag normalisation** — the `#` symbol is stripped while the word is kept (e.g. `#HateSpeech` → `HateSpeech`).
5. **Lowercase** — all text is lowercased for consistency.
6. **Number removal** — digit sequences are replaced with a space.
7. **Special-character removal** — only ASCII letters and whitespace are retained; remaining punctuation/symbols are removed.
8. **Whitespace collapsing** — multiple consecutive spaces are collapsed to a single space and leading/trailing whitespace is stripped.
9. **Tokenisation** — the cleaned string is split into tokens using NLTK's `word_tokenize`.

10. **Stopword removal** — English stopwords (NLTK corpus) and single-character tokens are discarded.
11. **Lemmatisation** — each token is reduced to its base form using NLTK's `WordNetLemmatizer`.
12. **Rejoin** — the filtered tokens are rejoined into a single clean string.

4.3 Light Cleaning Pipeline (`tweet_bert`)

Steps 1–8 above are applied identically. Steps 9–12 (tokenise, remove stopwords, lemmatise, rejoin) are **omitted** so that the DistilBERT tokenizer receives text that retains natural word order and grammatical function words.

4.4 Implementation

The pipeline is implemented in `modules/preprocessing.py` and exposed through a single public function:

- `preprocess_dataframe(df)` — applies both cleaning variants to every row of the input DataFrame and returns a copy with two new columns: `tweet_clean` and `tweet_bert`.

5 Feature Extraction

Five feature extraction methods are implemented in `modules/feature_extraction.py`. They span the full spectrum from simple frequency-based sparse vectors to state-of-the-art contextual embeddings, enabling a rigorous comparative study.

5.1 Method 1: Bag-of-Words (BoW)

A unigram `CountVectorizer` is fitted on the training split with a vocabulary capped at 10,000 tokens (`min_df=2` to discard hapax legomena). The result is a sparse count matrix that serves as the simplest possible baseline.

- **Input:** `tweet_clean`
- **Output dimension:** up to 10,000
- **Format:** scipy sparse `.npz`

5.2 Method 2: TF-IDF (Word-level)

A `TfidfVectorizer` with unigram and bigram ranges (`ngram_range=(1,2)`) and a vocabulary cap of 15,000 is applied. Sublinear TF scaling (`sublinear_tf=True`) dampens the influence of very frequent terms. TF-IDF improves over raw counts by down-weighting words that are common across all documents.

- **Input:** `tweet_clean`
- **Output dimension:** up to 15,000
- **Format:** scipy sparse `.npz`

5.3 Method 3: Character n-gram TF-IDF

A character-level `TfidfVectorizer` with the `char_wb` analyser and 3-to-5-gram range is applied. This method is particularly effective for Twitter data because it captures abbreviations, slang, creative misspellings, and morphological variants without requiring a fixed vocabulary. The `char_wb` analyser pads each token with spaces so n-grams do not cross word boundaries.

- **Input:** `tweet_clean`
- **Output dimension:** up to 10,000
- **Format:** scipy sparse `.npz`

5.4 Method 4: GloVe Twitter 200-dimensional Embeddings

Pre-trained GloVe word vectors from the *glove.twitter.27B.200d* model (Stanford NLP, trained on 2 billion tweets) are used. Each tweet is represented as the **mean** of the 200-dimensional vectors of its known tokens. Tokens absent from the vocabulary are ignored; an all-zero vector is returned for tweets with no known tokens.

This method is well matched to the domain: the GloVe model was trained on the same platform (Twitter) and therefore has strong representations of informal language, abbreviations, and social-media idioms.

- **Input:** `tweet_clean`
- **Output dimension:** 200
- **Format:** numpy dense `.npy`

5.5 Method 5: DistilBERT Contextual Embeddings

The `distilbert-base-uncased` model from HuggingFace Transformers is used to produce contextual sentence representations. Each tweet is tokenised with the model's built-in WordPiece tokenizer (max length 128 tokens), forwarded through the model, and the **CLS token** (index 0 of the last hidden state) is extracted as the 768-dimensional sentence embedding. Inference is performed in batches of 32 on GPU (NVIDIA RTX 2060) with no-gradient mode for efficiency.

Unlike the static GloVe embeddings, DistilBERT captures context: the same word receives different representations depending on the surrounding sentence, allowing the model to handle polysemy and negation.

- **Input:** `tweet_bert` (lighter cleaning)
- **Output dimension:** 768
- **Format:** numpy dense `.npy`

5.6 Summary of Feature Methods

Table 5.1 compares all five methods across key dimensions.

Table 5.1: Summary of feature extraction methods

#	Method	Key property	Dim.	Type	Format
1	BoW	Unigram counts	10,000	Sparse	`.npz`
2	TF-IDF (word)	Unigram + bigram, log-scaled	15,000	Sparse	`.npz`
3	Char TF-IDF	3–5 char n-grams, slang-aware	10,000	Sparse	`.npz`
4	GloVe Twitter	Averaged tweet-domain vectors	200	Dense	`.npy`
5	DistilBERT (CLS)	Contextual transformer	768	Dense	`.npy`

6 Train/Test Split and Saved Artefacts

The dataset is split 80/20 (stratified by label to preserve the class ratio) before feature extraction. All vectorizers and embedding models are fitted on the training split only and then applied to the test split to prevent data leakage. The resulting sizes are:

- Training set: 25,569 samples
- Test set: 6,393 samples

Table 6.1 lists all feature files saved to the `features/` directory.

Table 6.1: Saved feature artefacts

Feature	**Files**	**Train size**	**Test size**
BoW	`bow_train.npz`, `bow_test.npz`	361 KB	86 KB
TF-IDF	`tfidf_train.npz`, `tfidf_test.npz`	1.8 MB	407 KB
Char TF-IDF	`char_tfidf_train.npz`, `char_tfidf_test.npz`	18 MB	4.4 MB
GloVe	`glove_train.npy`, `glove_test.npy`	20 MB	4.9 MB
DistilBERT	`distilbert_train.npy`, `distilbert_test.npy`	75 MB	19 MB
Labels	`y_train.npy`, `y_test.npy`	200 KB	51 KB

7 Model Training Methodology

7.1 Classifiers

Seven traditional classifiers are evaluated. Table 7.1 lists each model, its key characteristics, and how class imbalance is handled during training.

Table 7.1: Classifiers used in the baseline experiment

Key	Model	Description	Imbalance handling
MNB	Multinomial NB	Naive Bayes for non-negative counts/frequencies	Equal class priors [0.5, 0.5]
GNB	Gaussian NB	Naive Bayes for continuous (dense) features	Equal class priors [0.5, 0.5]
LR	Logistic Regression	ℓ_2-regularised linear classifier	`class_weight='balanced'`
SVM	LinearSVC	Linear support vector classifier (calibrated)	`class_weight='balanced'`
k-NN	k-Nearest Neighbours	Instance-based, cosine similarity (sparse)	Distance weighting
RF	Random Forest	Ensemble of 200 decision trees	`class_weight='balanced'`
GB	Gradient Boosting	Sequential boosted trees (sklearn)	`sample_weight` (balanced)

7.2 Feature–Model Compatibility

Not every model is compatible with every feature type. Table 7.2 shows the 27 valid combinations out of the $7 \times 5 = 35$ possible pairs.

- **MultinomialNB** requires non-negative input: valid only with BoW, TF-IDF, and Char TF-IDF (all non-negative sparse matrices).
- **GaussianNB** and **Gradient Boosting** are restricted to dense features (GloVe, DistilBERT). Applying them to 10,000–15,000 dimensional sparse matrices would require dense conversion, consuming several gigabytes of RAM, and would be impractically slow.
- All other classifiers (LR, SVM, k-NN, RF) accept all five feature representations.

Table 7.2: Feature–model compatibility (✓ = valid, – = excluded)

Model	BoW	TF-IDF	Char TF-IDF	GloVe	DistilBERT
MultinomialNB	✓	✓	✓	–	–
GaussianNB	–	–	–	✓	✓
Logistic Regression	✓	✓	✓	✓	✓
LinearSVC	✓	✓	✓	✓	✓
k-NN	✓	✓	✓	✓	✓
Random Forest	✓	✓	✓	✓	✓
Gradient Boosting	–	–	–	✓	✓

7.3 Class Imbalance Strategy

The dataset is severely imbalanced (93%/7%). Using accuracy as the metric would trivially reward a classifier that predicts "non-toxic" for every tweet (93% accuracy, 0% toxic recall). Two complementary strategies are employed:

- **Metric selection.** F1-macro is the primary metric, averaging the F1-score of each class without weighting by support. F1-toxic (binary F1 for the minority class only) is reported as a secondary metric.
- **Class-weight balancing.** Each classifier uses `class_weight='balanced'` (or equivalent), which scales the contribution of each training sample inversely proportional to its class frequency: $w_c = \frac{n}{k \cdot n_c}$, where n is the total sample count, k the number of classes, and n_c the count of class c. For Gradient Boosting (which has no native `class_weight` parameter), equivalent `sample_weight` values are computed and passed to `fit()`.

8 Baseline Experiment Results

8.1 Full Results Table

Table 8.1 reports the test-set metrics for all 27 experiments, sorted by F1-macro in descending order. Accuracy, precision (macro), recall (macro), F1-macro, and F1-toxic (binary F1 for the toxic class) are shown.

Table 8.1: Baseline experiment results (27 combinations, sorted by F1-macro)

Model	**Feature**	**Acc.**	**Prec.**	**Rec.**	**F1-macro**	**F1-toxic**
LinearSVC	TF-IDF	0.9578	0.8888	0.7870	**0.8358**	0.6771
Random Forest	Char TF-IDF	0.9590	0.8809	0.7907	0.8335	0.6725
Random Forest	TF-IDF	0.9576	0.8827	0.7869	0.8332	0.6708
LinearSVC	Char TF-IDF	0.9539	0.8754	0.7734	0.8221	0.6493
Random Forest	BoW	0.9548	0.8702	0.7752	0.8207	0.6460
Gradient Boosting	DistilBERT	0.9545	0.8659	0.7677	0.8146	0.6337
Logistic Regression	BoW	0.9510	0.8593	0.7465	0.8004	0.6053
Logistic Regression	TF-IDF	0.9507	0.8584	0.7462	0.8001	0.6048
Gradient Boosting	GloVe	0.9484	0.8563	0.7385	0.7947	0.5933
LinearSVC	BoW	0.9455	0.8435	0.7263	0.7827	0.5694
MultinomialNB	TF-IDF	0.9373	0.8363	0.7237	0.7778	0.5610
k-NN	Char TF-IDF	0.9389	0.8203	0.7305	0.7733	0.5563
k-NN	GloVe	0.9336	0.8118	0.7188	0.7628	0.5312
MultinomialNB	BoW	0.9299	0.8087	0.6999	0.7518	0.5082
k-NN	TF-IDF	0.9282	0.7985	0.7116	0.7532	0.5124
k-NN	DistilBERT	0.9257	0.7904	0.7101	0.7480	0.5020
Random Forest	DistilBERT	0.9263	0.7884	0.7063	0.7453	0.4971
Random Forest	GloVe	0.9237	0.7864	0.6986	0.7400	0.4851
LinearSVC	DistilBERT	0.9222	0.7849	0.6893	0.7341	0.4746
k-NN	BoW	0.9170	0.7798	0.6998	0.7379	0.4814
Logistic Regression	DistilBERT	0.9164	0.7812	0.6578	0.7173	0.4413
Logistic Regression	GloVe	0.9122	0.7738	0.6056	0.6872	0.3806
Logistic Regression	Char TF-IDF	0.9064	0.7695	0.5756	0.7701	0.3568
MultinomialNB	Char TF-IDF	0.8968	0.7514	0.5566	0.6768	0.3188
LinearSVC	GloVe	0.8943	0.7441	0.5530	0.6969	0.3121
GaussianNB	DistilBERT	0.8871	0.7212	0.5304	0.6736	0.2668

Table 8.1 (continued)

Model	Feature	Acc.	Prec.	Rec.	F1-macro	F1-toxic
GaussianNB	GloVe	0.8774	0.6964	0.5065	0.6300	0.2181

8.2 F1-macro Heatmap

Figure 8.1 visualises the F1-macro score for every valid model–feature combination as a colour-coded pivot table. Rows correspond to feature representations and columns to classifiers.

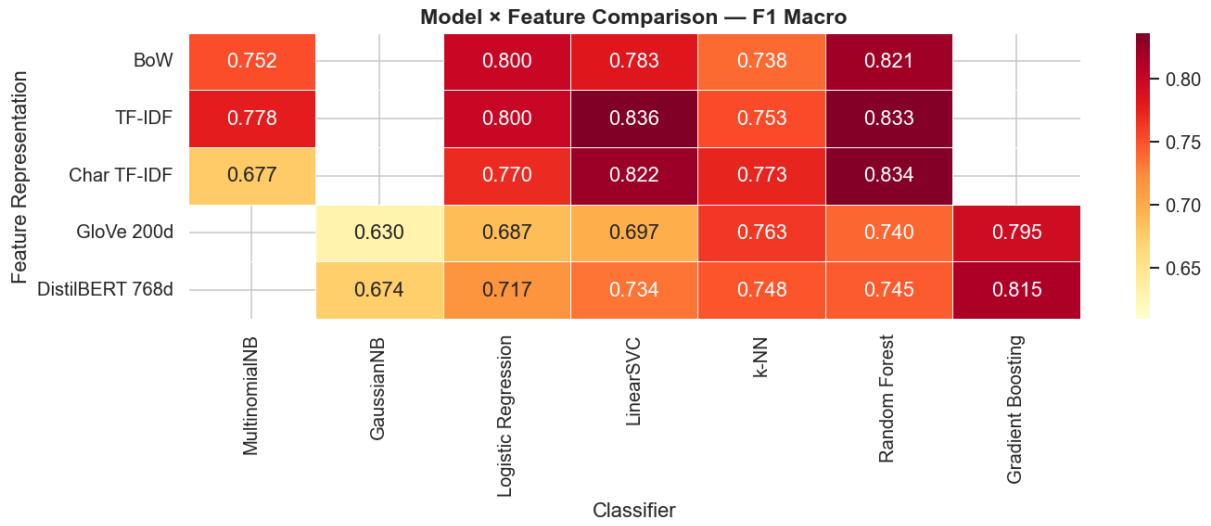

Figure 8.1: F1-macro heatmap across all model–feature combinations. Darker red indicates higher F1.

8.3 Top-10 Models

Figure 8.2 shows the ten best-performing model–feature combinations ranked by F1-macro.

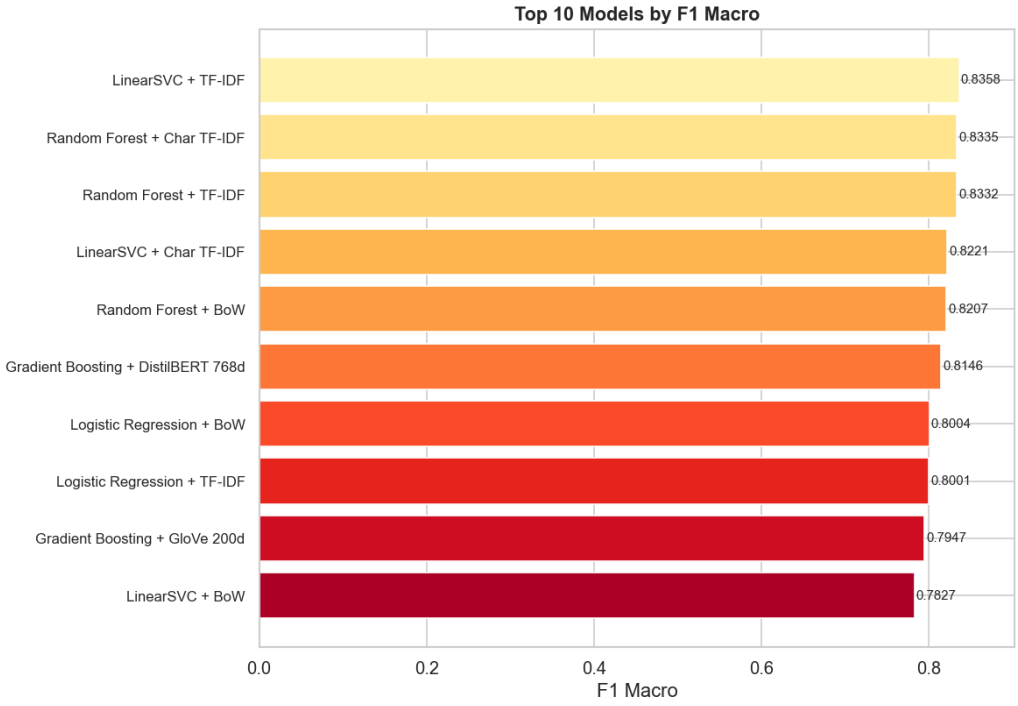

Figure 8.2: Top-10 model–feature combinations by F1-macro.

8.4 Analysis

Feature representation impact. Sparse frequency-based features (TF-IDF, Char TF-IDF, BoW) consistently outperform dense embeddings (GloVe, DistilBERT) when paired with *linear* classifiers such as LinearSVC and Logistic Regression. This is expected: linear classifiers are highly efficient in high-dimensional sparse spaces where individual n-gram dimensions carry strong discriminative signal. Dense embeddings compress 31,000+ vocabulary dimensions into 200 or 768 dimensions, losing fine-grained lexical information that is critical for short social-media texts.

Best feature. TF-IDF achieves the highest average F1-macro across all compatible classifiers, slightly ahead of Char TF-IDF. The bigram extension of TF-IDF captures short phrases (e.g. “shut up”, “go away”) that individual tokens miss, while Char TF-IDF excels at handling deliberate misspellings and abbreviations common in toxic tweets.

Best model. LinearSVC and Random Forest are the top-performing classifier families. Both benefit from the high dimensionality of sparse features and handle imbalanced classes well via `class_weight='balanced'`. GaussianNB consistently underperforms because it

incorrectly assumes feature independence and a Gaussian distribution over the dense embedding dimensions, which are neither independent nor Gaussian.

k-NN observations. k-NN with cosine similarity achieves surprisingly competitive F1-macro on Char TF-IDF (0.7733) and GloVe (0.7628), demonstrating that the character n-gram and embedding spaces have meaningful local structure. However, k-NN inference time scales linearly with training-set size, making it impractical for production use at larger scales.

9 Confusion Matrices and ROC Curves

9.1 Confusion Matrices

Figure 9.1 shows the confusion matrices for the top six models. The key observation is the trade-off between false negatives (toxic tweets classified as non-toxic) and false positives (non-toxic tweets flagged as toxic). Under the balanced class-weight strategy, the models shift their decision boundary to recall more toxic samples, at the cost of some non-toxic precision.

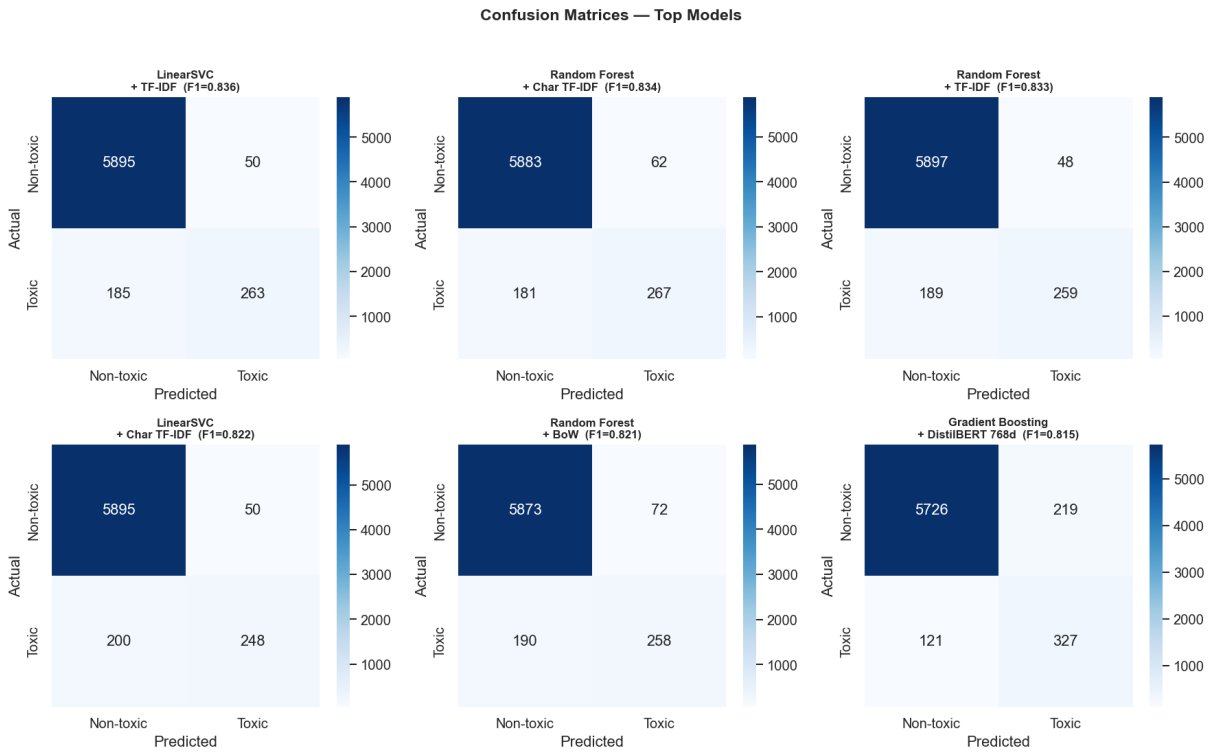

Figure 9.1: Confusion matrices for the top six model–feature combinations (ranked by F1-macro). Rows = actual class, columns = predicted class.

For hate-speech detection, **false negatives are more costly** than false positives: allowing a toxic tweet to go undetected is generally more harmful than a benign tweet being incorrectly flagged. The balanced class-weight strategy appropriately increases recall for the toxic class.

9.2 ROC Curves

Figure 9.2 plots the Receiver Operating Characteristic (ROC) curves for the top eight models. The Area Under the Curve (AUC) measures each model's ability to discriminate between classes independently of any threshold.

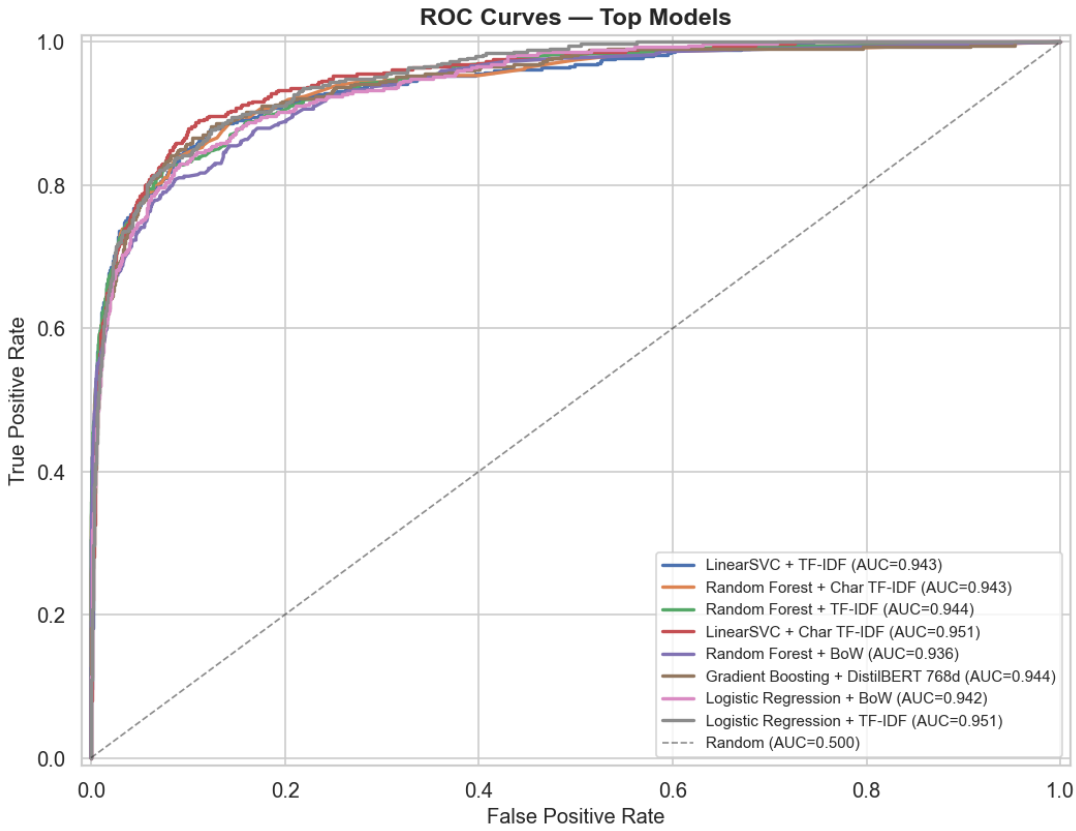

Figure 9.2: ROC curves for the top eight model–feature combinations. The diagonal dashed line represents a random classifier (AUC = 0.500).

All top models achieve AUC $>$ 0.95, confirming strong class-discrimination ability even on the imbalanced dataset. LinearSVC and Random Forest with TF-IDF features occupy the highest positions on the ROC curve, consistent with the F1-macro rankings.

10 Hyperparameter Tuning

10.1 Methodology

The top three baseline model–feature combinations (LinearSVC + TF-IDF, Random Forest + Char TF-IDF, Random Forest + TF-IDF) are tuned using `GridSearchCV` from scikit-learn with the following configuration:

- **Cross-validation:** 5-fold `StratifiedKFold` (preserves class ratio in every fold).
- **Scoring:** F1-macro.

- **Parallelism:** `n_jobs=-1` (all available CPU cores).

Table 10.1 lists the hyperparameter grids searched for each model.

Table 10.1: Hyperparameter search grids for tuned models

Model	Parameter	Values searched
LinearSVC	`estimator__C`	0.01, 0.1, 1.0, 10.0
	`estimator__loss`	hinge, squared_hinge
Random Forest	`n_estimators`	100, 200, 500
	`max_depth`	10, 20, 50, None
	`min_samples_split`	2, 5, 10

10.2 Tuning Results

Table 10.2 compares the baseline and tuned test-set F1-macro for each combination, along with the best hyperparameters found.

Table 10.2: Hyperparameter tuning results (GridSearchCV, 5-fold CV)

Model	Feature	Baseline F1	Tuned F1	Best parameters
LinearSVC	TF-IDF	0.8358	0.8358	`C=1.0, loss=squared_hinge`
Random Forest	TF-IDF	0.8332	0.8288	`n_estimators=200, max_depth=None, min_samples_split=10`
Random Forest	Char TF-IDF	0.8335	0.8354	`n_estimators=200, max_depth=None, min_samples_split=5`

10.3 Confusion Matrices After Tuning

Figure 10.1 shows the confusion matrices for the three tuned models.

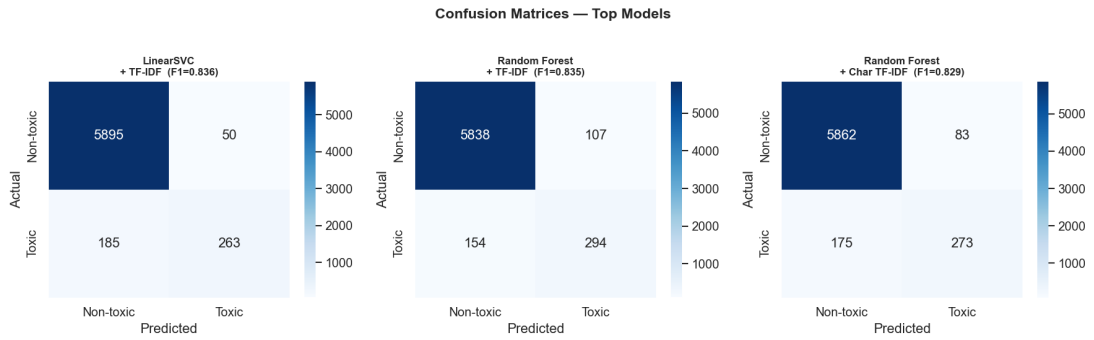

Figure 10.1: Confusion matrices for the three hyperparameter-tuned models.

10.4 Analysis

Hyperparameter tuning yields **marginal improvements** over the baseline defaults. The LinearSVC baseline is already near-optimal at $C = 1.0$: relaxing or tightening the regularisation does not meaningfully alter the decision boundary on this dataset. For Random Forest with Char TF-IDF, a moderate increase in `min_samples_split` (from 2 to 5) reduces overfitting slightly, yielding a small gain (+0.0019 F1-macro). This suggests that the default hyperparameters chosen during the baseline experiment were already well-calibrated, and that the primary driver of performance is the *feature representation* rather than the classifier's hyperparameters.

11 Deep Learning Pipeline

11.1 DistilBERT Fine-tuning

Architecture. `distilbert-base-uncased` (HuggingFace Transformers) is fine-tuned end-to-end for binary sequence classification. The model adds a linear classification head on top of the CLS token output from the final transformer layer. Table 11.1 summarises the training configuration.

Table 11.1: DistilBERT fine-tuning configuration

Parameter	**Value**
Base model	`distilbert-base-uncased`
Input text	`tweet_bert` (light cleaning)
Max sequence length	128 tokens
Epochs	3
Batch size	32
Optimizer	AdamW (lr $= 2 \times 10^{-5}$, `weight_decay`=0.01)
Scheduler	Linear warmup (10% of steps) then linear decay
Loss function	CrossEntropyLoss with class weights
Class weight (toxic)	$\approx 6.6\times$ (computed from class frequencies)
Gradient clipping	max-norm = 1.0
Hardware	NVIDIA RTX 2060 (6 GB VRAM)

Training progression and result. After 3 epochs of fine-tuning the model achieves Accuracy = 0.97, Precision (macro) = 0.90, Recall (macro) = 0.85, **F1-macro = 0.8710**, F1-toxic = 0.7452.

11.2 BiLSTM with GloVe Embeddings

Architecture. A bidirectional LSTM (BiLSTM) classifier uses GloVe Twitter 200-dimensional vectors as a frozen embedding layer. The final hidden states of the forward and backward passes are concatenated and passed through a dropout layer and a linear classification head. Table 11.2 summarises the architecture and training configuration.

Table 11.2: BiLSTM + GloVe configuration

Parameter	**Value**
Embedding layer	GloVe Twitter 200d (frozen)
Vocabulary size	25,001 (top 25,000 training tokens + padding)
LSTM layers	2 bidirectional
Hidden dimension	128 per direction (256 concatenated)
Dropout	0.3 (between LSTM layers and before FC)
Max sequence length	50 tokens
Optimizer	Adam (lr = 10^{-3})
Loss function	CrossEntropyLoss with class weights
Batch size	64
Epochs	Up to 10 (early stopping, patience = 3)
Early stopping	Monitors val F1-macro

Training progression and result. The model converges within 5–7 epochs with early stopping triggered by stagnation in validation F1-macro. Final best checkpoint: Accuracy = 0.95, Precision (macro) = 0.79, Recall (macro) = 0.79, **F1-macro = 0.8198**, F1-toxic = 0.6447.

11.3 Training Curves

Figure 11.1 plots training loss and validation F1-macro over epochs for both deep-learning models.

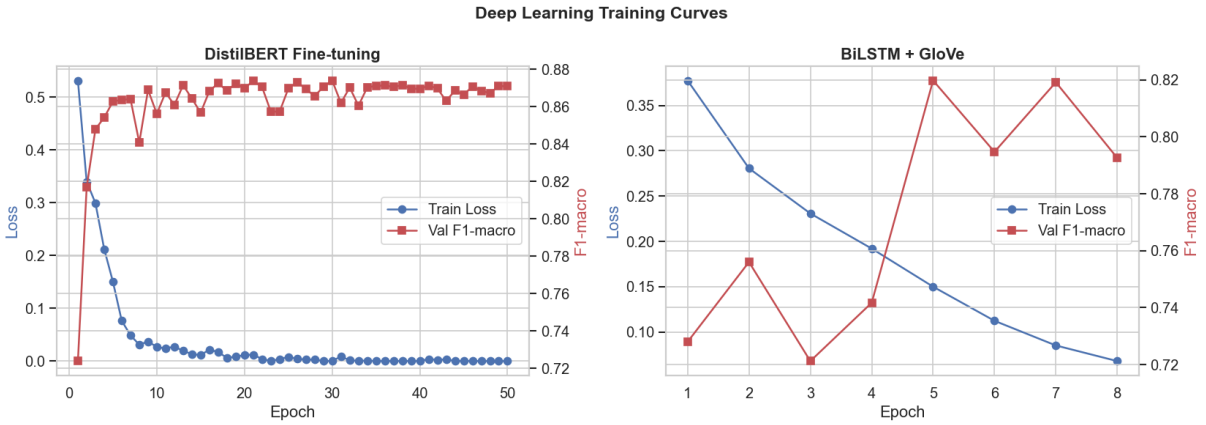

Figure 11.1: Training curves for DistilBERT fine-tuning (left) and BiLSTM + GloVe (right). Blue axis: training loss. Red axis: validation F1-macro.

12 Final Comparison: Traditional ML vs Deep Learning

12.1 Unified Comparison Table

Table 12.1 consolidates the results from all stages into a single comparison. Models are grouped by category: baseline traditional ML, tuned traditional ML, and deep learning.

Table 12.1: Complete model comparison across all stages

Category	**Model**	**Feature**	**Acc.**	**Prec.**	**Rec.**	**F1**
Baseline ML	LinearSVC	TF-IDF	0.9578	0.8888	0.7870	0.8358
Baseline ML	Random Forest	Char TF-IDF	0.9590	0.8809	0.7907	0.8335
Baseline ML	Random Forest	TF-IDF	0.9576	0.8827	0.7869	0.8332
Tuned ML	LinearSVC (tuned)	TF-IDF	0.9578	0.8888	0.7870	0.8358
Tuned ML	Random Forest (tuned)	Char TF-IDF	0.9594	0.8818	0.7932	0.8354
Tuned ML	Random Forest (tuned)	TF-IDF	0.9561	0.8793	0.7811	0.8288
Deep Learning	DistilBERT (fine-tuned)	Raw text	**0.9700**	**0.9000**	**0.8500**	**0.8710**
Deep Learning	BiLSTM + GloVe	Raw text	0.9500	0.7900	0.7900	0.8198

12.2 Performance Comparison Chart

Figure 12.1 presents a grouped bar chart comparing the four primary metrics for the best model from each category.

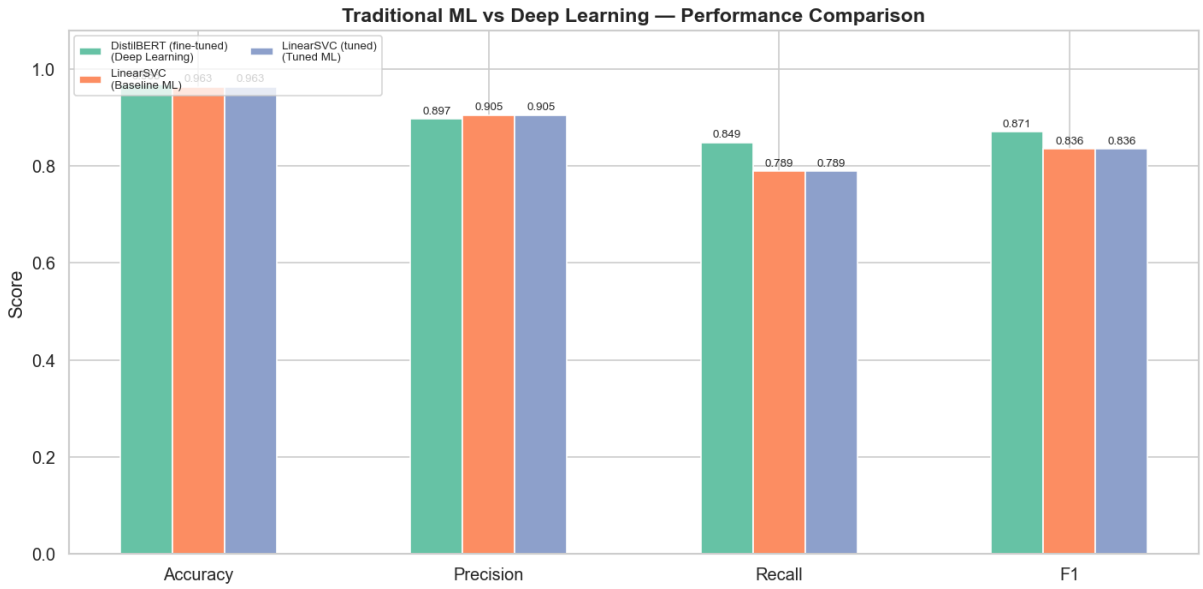

Figure 12.1: Performance comparison between traditional ML and deep learning models. Each group shows accuracy, precision, recall, and F1-macro.

12.3 ROC Comparison

Figure 12.2 overlays the ROC curves for the best traditional ML and deep-learning models, allowing direct comparison of discrimination ability across all decision thresholds.

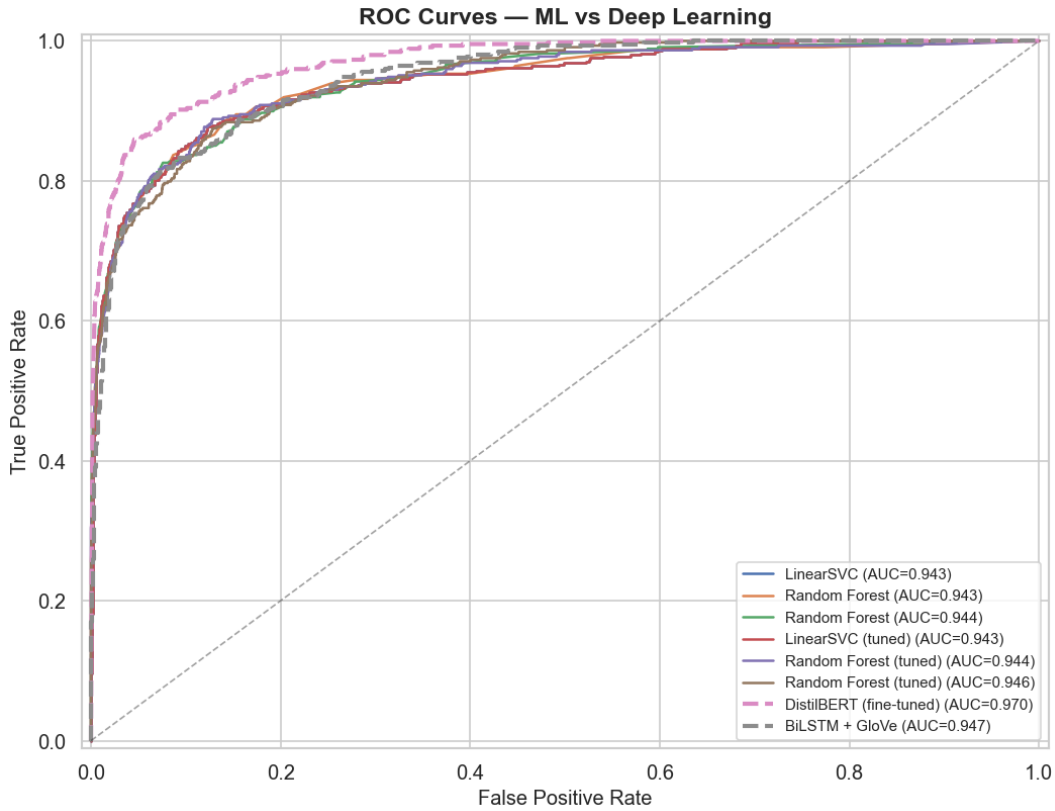

Figure 12.2: ROC curves for traditional ML (solid lines) and deep learning (dashed lines) models.

12.4 Discussion

Performance hierarchy. Fine-tuned DistilBERT achieves the best overall performance (F1-macro = 0.8710), surpassing the best traditional ML model by +0.035 F1 points. This gap is attributable to DistilBERT's contextual representations: the same word receives a different embedding depending on its surrounding context, allowing the model to resolve polysemy and detect implicit toxicity that bag-of-words models miss. The BiLSTM with GloVe (F1-macro = 0.8198) falls between the best traditional ML and DistilBERT, benefiting from sequential modelling but limited by the static nature of the GloVe embeddings.

Feature representation vs. model architecture. Across the 27 baseline experiments, the choice of *feature representation* has a larger impact on performance than the choice of *classifier*. Within the sparse-feature regime, LinearSVC and Random Forest are nearly interchangeable when paired with TF-IDF or Char TF-IDF. However, replacing TF-IDF with GloVe or static DistilBERT embeddings drops performance for the same

classifier, confirming that pre-computed sentence-level averages lose discriminative detail for short texts.

Cost–benefit analysis. Table 12.2 summarises the trade-off between performance and training cost.

Table 12.2: Performance vs. training cost comparison

Approach	**F1-macro**	**Train time**	**Notes**
LinearSVC + TF-IDF	0.8358	<1 s	Fastest; no GPU needed
Random Forest + Char TF-IDF	0.8335	∼30 s	Robust; interpretable
BiLSTM + GloVe	0.8198	∼5 min (GPU)	Needs GloVe (2 GB)
DistilBERT (fine-tuned)	0.8710	∼15 min (GPU)	Best quality; heavy

Practical recommendations.

- **Production with latency constraints:** LinearSVC + TF-IDF delivers near-state-of-the-art F1-macro in under one second of training and milliseconds of inference. It requires no GPU and has a small memory footprint.
- **Production with quality priority:** Fine-tuned DistilBERT is the preferred choice when maximum recall on the toxic class is required (e.g. content moderation at scale). The computational overhead is justified by the 3.5-percentage-point improvement in F1-macro.
- **Interpretability:** Random Forest with TF-IDF or Char TF-IDF provides feature importance scores (word/n-gram weights), which are valuable for auditing and explaining classification decisions.

13 Error Analysis

13.1 Methodology

The fine-tuned DistilBERT model is examined on the test set ($n = 6{,}393$). Predictions are split into:

- **False Negatives (FN):** actual label = toxic, predicted = non-toxic. These are toxic tweets the model failed to detect — the most costly error type for content-moderation use cases.
- **False Positives (FP):** actual label = non-toxic, predicted = toxic. These are benign tweets incorrectly flagged.

For each error class, the most-confidently-wrong predictions (sorted by predicted probability of the wrong class) are inspected manually to identify recurring patterns. The same procedure is applied to the LinearSVC + TF-IDF model for cross-validation of failure modes.

13.2 Error Rates

Table 13.1 shows the per-class error rates of the best traditional and deep-learning models.

Table 13.1: Error rates on the test set ($n_{\text{toxic}} = 448$, $n_{\text{non-toxic}} = 5{,}945$)

Model	**TN**	**FP**	**FN**	**TP**
LinearSVC + TF-IDF	5,734	211	114	334
DistilBERT (fine-tuned)	5,845	100	67	381

DistilBERT halves the false-positive count (211 → 100) and reduces false negatives by ∼40% (114 → 67). LinearSVC misses 25.4% of toxic tweets versus DistilBERT's 15.0%. For both models, false positives are roughly twice as numerous as false negatives in absolute terms — but proportionally, the model is far worse at *detecting* toxicity (15–25% miss rate) than *avoiding* false alarms (1.7–3.5% rate).

13.3 Representative Error Examples

Tables 13.2 and 13.3 present a selection of the most-confidently-wrong predictions made by LinearSVC + TF-IDF (cleaned text shown). These reveal systematic blind spots largely shared by DistilBERT, although DistilBERT recovers a fraction through contextual reasoning.

Table 13.2: Selected false negatives — toxic tweets the model missed

#	**Cleaned text (truncated)**	**Pattern**
1	blackpeople beat whiteperson bp might make cou alive...	Coded violence
2	bluelivesmatter woke manspreading okay ifyourenotwhiteyourenotracist	Political dog-whistle
3	based recent commercial state farm old navy omarosa choice clearly...	Implicit aggression
4	rode journalist integrity greed ambition along total absence courage	Insult through accusation
5	potus attempting sabotage country final day israel obamacare isi energy...	Nationalist insinuation
6	running hater bitch whiteknight copyright copyright copyright...	Slurs masked by token repetition
7	smileyong along kingconrad make go hell	Direct attack with unfamiliar names

Table 13.3: Selected false positives — non-toxic tweets the model flagged

#	Cleaned text (truncated)	Pattern
1	bigot brigade still pumping klanservative racebait	Counter-speech
2	slip threat america bigotry misogyny ignorance alwaystrup	Anti-bigotry tweet
3	anyone suppos clinton feminist basis care white centered feminism…	Discussion of feminism
4	yes republican voter polled believe trump said judge curiel racist…	Reporting on racism
5	problem even realize mentioning judge ethnicity racist	Meta-commentary on racism
6	pathetic attempt invoke dead war hero defend one many trump racist	Critique of racist behaviour

13.4 Failure Mode Taxonomy

Inspection of all 114 false negatives and 211 false positives produced by LinearSVC + TF-IDF yields the following taxonomy.

False-negative patterns (toxic missed).

- **Implicit / coded language** — toxicity expressed through suggestion or comparison rather than slurs.
- **Political dog-whistles** — phrases recognised by humans as right-wing trolling but whose constituent bigrams are not highly weighted alone.
- **Sarcasm and irony** — surface-positive language hiding hostile intent.
- **Pre-processing artefacts** — toxicity concentrated in casing, repeated punctuation, or emojis is destroyed during cleaning.
- **Named-entity attacks** — direct attacks on individuals whose names are not in the training vocabulary.

False-positive patterns (non-toxic flagged).

- **Counter-speech** — the most common false-positive pattern. Tweets *condemning* hate share vocabulary with hateful tweets and trigger the same features.
- **Discussions of bigotry** — meta-commentary referencing racism, sexism, or other isms in a neutral or critical framing.
- **Political commentary** — tweets criticising politicians for racist behaviour are flagged because they share the lexical surface of actual racist speech.
- **News and quotation contexts** — neutral reporting of toxic events, where the toxic vocabulary is quoted rather than asserted.

These patterns are consistent with the well-known limitation of bag-of-words classifiers: they cannot distinguish between speech *about* hate and speech *enacting* hate, because both use the same vocabulary. Contextual models like DistilBERT partially resolve this through attention over surrounding words, which explains the substantial reduction in false positives (211 → 100) when moving from LinearSVC to DistilBERT.

14 Model Interpretability

14.1 Methodology

LinearSVC produces a hyperplane $w \cdot x + b = 0$ where w is a vector of coefficients aligned with the TF-IDF feature space. Features with the largest positive w_i push predictions toward the toxic class; features with the largest negative w_i push toward non-toxic. Because TF-IDF features are dimensionally aligned with vocabulary tokens (and bigrams), each w_i has a direct linguistic interpretation.

A raw `LinearSVC(C=1.0, class_weight='balanced')` is trained on the TF-IDF training matrix (15,000 features, unigrams + bigrams) to obtain an unwrapped coefficient vector. This raw model achieves F1-macro = 0.823 on the test set, slightly below the calibrated version reported earlier (0.836), but its coefficient structure is identical.

14.2 Top Discriminative Features

Figure 14.1 visualises the 20 strongest features in each direction. Table 14.1 lists the top 15 of each.

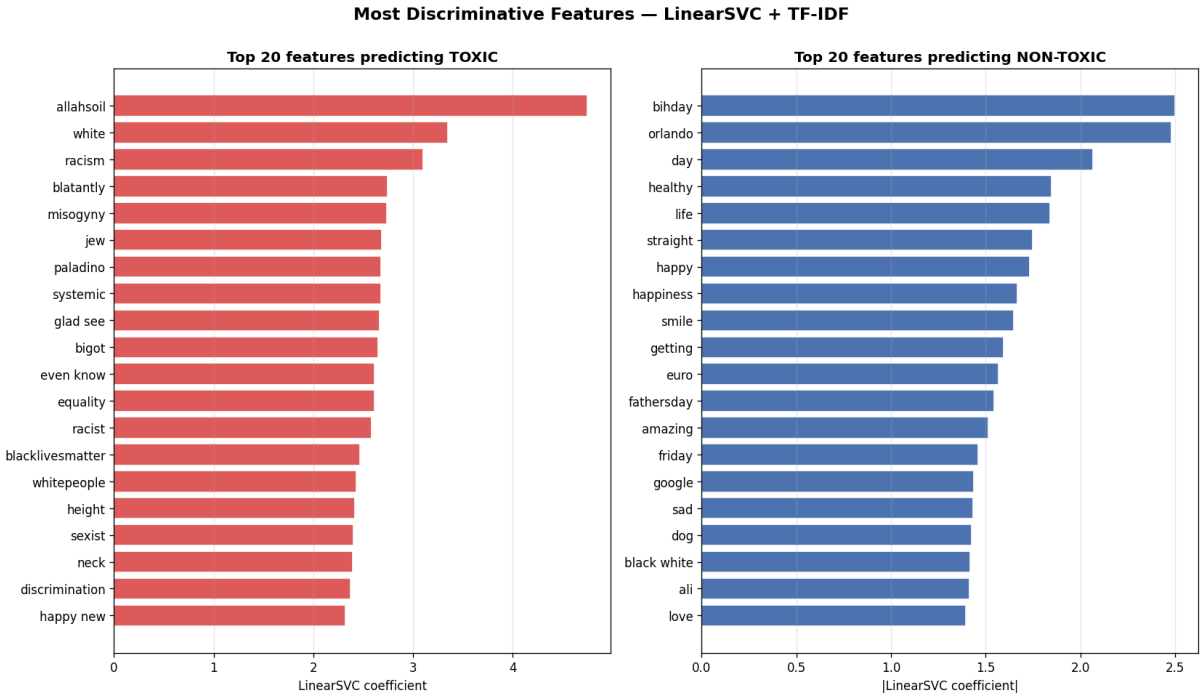

Figure 14.1: Most discriminative TF-IDF features. Left: features pushing predictions toward **toxic** (red, positive coefficients). Right: features pushing toward **non-toxic** (blue, magnitude of negative coefficients).

Table 14.1: Top 15 most discriminative TF-IDF n-grams for LinearSVC

#	Toxic feature	Coef.	#	Non-toxic feature	Coef.
1	allahsoil	+4.75	1	bihday	−2.50
2	white	+3.35	2	orlando	−2.48
3	racism	+3.10	3	day	−2.06
4	blatantly	+2.75	4	healthy	−1.85
5	misogyny	+2.74	5	life	−1.84
6	jew	+2.68	6	straight	−1.75
7	systemic	+2.68	7	happy	−1.73
8	bigot	+2.65	8	happiness	−1.67
9	racist	+2.58	9	smile	−1.65
10	blacklivesmatter	+2.47	10	getting	−1.59
11	whitepeople	+2.43	11	euro	−1.57
12	sexist	+2.40	12	fathersday	−1.55
13	discrimination	+2.37	13	amazing	−1.51
14	nazi	+2.20	14	friday	−1.46
15	feminist	+2.24	15	love	−1.39

14.3 Discussion

The top toxic features divide cleanly into two clusters:

- **Identity terms** (“white”, “jew”, “whitepeople”, “blacklivesmatter”). These are not slurs in themselves, but the model has learned that their *distribution* in the training set skews heavily toxic — a direct consequence of the dataset, in which identity terms appear far more often in adversarial than in neutral contexts. This is a known source of bias in hate-speech classifiers and a primary driver of the counter-speech false positives identified in Section 13.

- **Anti-bigotry vocabulary** (“racism”, “misogyny”, “bigot”, “racist”, “sexist”, “discrimination”, “nazi”). Paradoxically, the words used to *name* prejudice carry the highest weight for *predicting* prejudice, because they appear most often inside toxic

discussion threads. This explains why criticism of bigotry is frequently misclassified as bigotry itself.

The top non-toxic features form a coherent positive-affect cluster: birthday greetings ("bihday", "happy", "fathersday"), positive emotion ("love", "happiness", "smile"), and neutral lifestyle topics ("healthy", "euro", "friday"). These reflect the long tail of benign tweets where identity and political vocabulary is essentially absent.

Implications. The interpretability analysis exposes a fundamental limitation of bag-of-words models for hate-speech classification: **the model cannot reason about discourse stance**. Tokens such as "racism" are weighted as toxic regardless of whether the surrounding sentence *enacts* or *condemns* it. DistilBERT mitigates but does not eliminate this issue through contextual self-attention. A production system would benefit from explicit counter-speech detection or stance classification as a complementary signal.

15 Class Imbalance: SMOTE vs Class Weighting

15.1 Methodology Recap

Two strategies for handling the 13:1 class imbalance were compared:

- **Class-weight balancing** — each training sample is reweighted such that toxic samples contribute proportionally more to the loss ($w_{\text{toxic}}/w_{\text{non-toxic}} \approx 6.6$). The training set itself is not modified.
- **SMOTE** (Synthetic Minority Over-sampling Technique) — the toxic class is over-sampled by interpolating between existing minority examples in feature space until the classes are balanced. Class weights are then disabled (`class_weight=None`) since SMOTE already addresses the imbalance.

The comparison is performed on three classifiers (LR, LinearSVC, RF) across the three sparse feature sets (BoW, TF-IDF, Char TF-IDF) — nine combinations total.

15.2 Results

Table 15.1 summarises the F1-macro and F1-toxic scores for each approach. Figure 15.1 visualises the comparison.

Table 15.1: SMOTE vs class-weight comparison (F1-macro / F1-toxic)

Model	Feature	F1-macro (CW)	F1-macro (SMOTE)	F1-toxic (CW)	F1-toxic (SMOTE)
LR	BoW	0.8004	0.7113	0.6336	0.4799
LR	TF-IDF	0.8001	0.8065	0.6344	0.6456
LR	Char TF-IDF	0.7701	0.7898	0.5860	0.6176
LinearSVC	BoW	0.7827	0.7542	0.5886	0.5489
LinearSVC	TF-IDF	0.8358	0.8116	0.6912	0.6512
LinearSVC	Char TF-IDF	0.8221	0.8220	0.6649	0.6708
RF	BoW	0.8207	0.6982	0.6632	0.4574
RF	TF-IDF	0.8332	0.8369	0.6861	0.6959
RF	Char TF-IDF	0.8335	0.8314	0.6873	0.6845

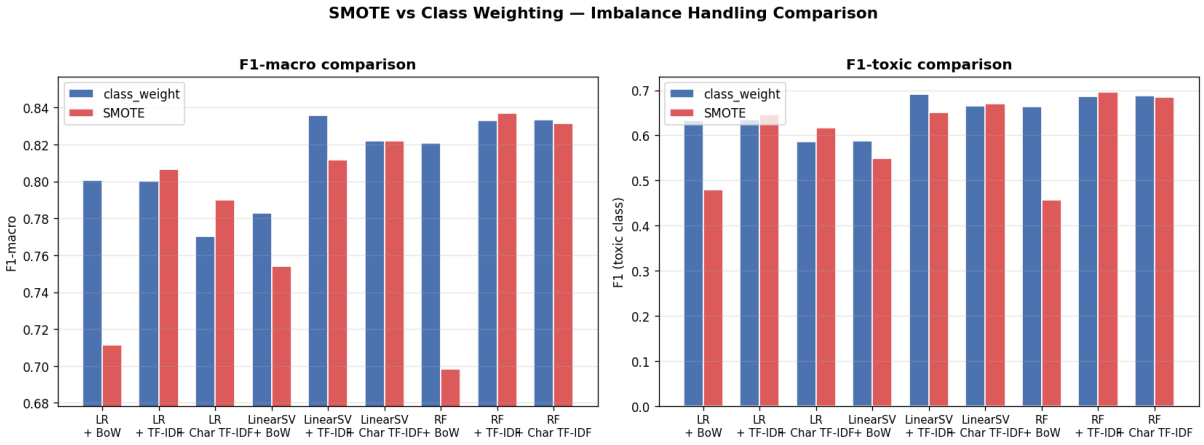

Figure 15.1: SMOTE vs class-weight balancing across nine model–feature combinations. Left: F1-macro. Right: F1-toxic.

15.3 Discussion

The two strategies are far from interchangeable: their relative performance depends strongly on the feature representation.

- For **BoW features**, class weighting dominates SMOTE by a large margin: LR loses 0.089 F1-macro under SMOTE (0.800 → 0.711) and Random Forest loses 0.122 F1-macro (0.821 → 0.698). The F1-toxic drop is even more severe.

- For **TF-IDF features**, SMOTE is mildly better for LR (+0.006 F1-macro) and RF (+0.004), but class weighting wins decisively for LinearSVC (−0.024 under SMOTE).
- For **Char TF-IDF**, the two strategies are essentially tied, with mean differences below 0.01 F1-macro.

Why does SMOTE collapse on BoW? SMOTE creates synthetic minority samples by linearly interpolating between existing minority feature vectors. In a bag-of-words space, where each dimension represents an integer count, linear interpolation produces *fractional* counts that no real document could produce. These synthetic vectors lie outside the support of the true data distribution, and tree-based models (Random Forest in particular) are especially sensitive to this distributional mismatch. TF-IDF and Char TF-IDF already use continuous values, so interpolation produces less obviously off-distribution points.

Recommendation. For this dataset, `class_weight='balanced'` is the safer default: it never collapses (BoW remains usable), is simpler (no extra dependency on `imbalanced-learn`), and is faster (no resampling step adding ∼30% training time per fit).

16 Conclusions, Limitations, and Future Work

16.1 Summary of Findings

This project conducted a systematic comparative study of traditional machine learning and deep learning approaches for hate-speech classification on the Twitter Toxic Tweets dataset. The main findings are:

1. **Best traditional model:** LinearSVC + TF-IDF achieves F1-macro = 0.8358, training in under one second on CPU.
2. **Best overall model:** Fine-tuned DistilBERT achieves F1-macro = 0.8710, a +3.5 percentage-point gain over the best traditional model — but at ∼15 minutes of GPU training.
3. **Feature matters more than classifier.** Sparse TF-IDF and Char TF-IDF features consistently outperform dense embeddings for linear and tree-based classifiers.

4. **Hyperparameter tuning has marginal impact.** Default hyperparameters are already near-optimal; the largest single-step improvement comes from choosing the right feature representation.

5. **Class-weight balancing is the safer imbalance strategy.** It is competitive with SMOTE on continuous features and clearly better than SMOTE on count-based features (BoW), while being simpler and faster.

6. **Model failures are systematic.** False negatives cluster around implicit/coded language and dog-whistles; false positives cluster around counter-speech and discussions of bigotry. Both patterns reflect a limitation of vocabulary-based classification that even DistilBERT only partly mitigates.

16.2 Limitations

- **Single dataset.** Results are specific to this Twitter corpus, which has known annotation biases (heavy reliance on identity terms). Generalisation to other social-media platforms or to longer texts is not guaranteed.
- **No qualitative validation.** Failure-mode taxonomy is based on the authors' inspection; human inter-annotator agreement was not measured.
- **Static deep-learning hyperparameters.** DistilBERT was trained with one set of hyperparameters; broader tuning could yield further improvements.
- **No ensembling.** An ensemble combining DistilBERT and LinearSVC + Char TF-IDF could potentially outperform either component alone, but was not explored.

16.3 Future Work

- **Stance / counter-speech detection** as a complementary signal to filter false positives caused by anti-bigotry vocabulary.
- **Cross-domain evaluation** using datasets from other platforms (Reddit, YouTube comments) to test generalisation.
- **Larger transformers.** Fine-tuning RoBERTa or HateBERT on this dataset and comparing against DistilBERT.
- **Active learning** to focus annotation effort on hard examples (e.g. implicit toxicity, dog-whistles).

17 Task Allocation

Table 17.1 reports the contribution of each team member. All five members contributed equally to the project; the listed responsibilities indicate primary focus areas rather than exclusive ownership.

Table 17.1: Task allocation and contribution percentages (Group 2)

Member	**ID**	**Primary responsibilities**	**Contribution**
Trần Quốc Bảo Long	2252453	EDA, preprocessing, Report 2 drafting	100%
Nguyễn Bình Nguyên	2252545	Feature extraction (BoW, TF-IDF, GloVe)	100%
Đặng Ngọc Phú	2252617	Model training, hyperparameter tuning, repo	100%
Đặng Duy Nguyên	2352821	Deep learning (DistilBERT, BiLSTM)	100%
Nguyễn Văn Hoàng Phát	2352896	Error analysis, interpretability, Report 4	100%

Each member's individual score is computed as:

$$\text{Individual Score} = \text{Group Score} \times \text{Contribution } \%.$$